

《数据科学》实验报告

班级	003	姓名	史润	学号	20241220
实验题目	Python 数据处理				
实验时间	2025-10-25	实验地点	DS1408		
学年学期	2025-2026(1)	实验性质	☐ 验证性 ☐ 设计性 ☒ 综合性		

1. Iris 数据集探索与异常值检测

① 加载并初步查看 Iris 数据集

代码:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from sklearn.datasets import load_iris
# 加载 Iris 数据集
iris = load_iris()
# 打印 iris.data 的前 5 行
print("iris.data 的前 5 行: ")
print(iris.data[:5])
print()
```

结果:

```
iris.data的前5行:
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
```

```
# 打印 iris.target 的后 5 行
print("iris.target 的后 5 行: ")
print(iris.target[-5:])
print()
```

结果:

```
iris.target的后5行:
[2 2 2 2 2]
```

```
# 打印 iris.feature_names
print("iris.feature_names: ")
print(iris.feature_names)
```

```
iris.feature_names:
['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
```

打印数据描述的前 300 个字符

```
print("iris.DESCR 的前 300 个字符: ")
```

```
print(iris.DESCR[:300])
```

结果:

```
iris.DESCR的前300个字符:
.. _iris_dataset:

Iris plants dataset
-----

**Data Set Characteristics:**

: Number of Instances: 150 (50 in each of three classes)
: Number of Attributes: 4 numeric, predictive attributes and the class
: Attribute Information:
  - sepal length in cm
  - sepal width in cm
  - pet
```

② 创建一个包含特征和标签的 pandas DataFrame

```
df = pd.DataFrame(data=iris.data, columns=iris.feature_names)
```

```
df["target"] = iris.target # 新增'target'列存储标签
```

```
column_mapping = {
```

```
    "sepal length (cm)": "花萼长度",
```

```
    "sepal width (cm)": "花萼宽度",
```

```
    "petal length (cm)": "花瓣长度",
```

```
    "petal width (cm)": "花瓣宽度",
```

```
} # 重命名特征列为中文
```

```
df.rename(columns=column_mapping, inplace=True)
```

打印前 5 行验证

```
print("DataFrame 前 5 行: ")
```

```
print(df.head())
```

结果:

```
DataFrame前5行:
   花萼长度  花萼宽度  花瓣长度  花瓣宽度  target
0     5.1     3.5     1.4     0.2         0
1     4.9     3.0     1.4     0.2         0
2     4.7     3.2     1.3     0.2         0
3     4.6     3.1     1.5     0.2         0
4     5.0     3.6     1.4     0.2         0
```

③ 特征量的分布分析

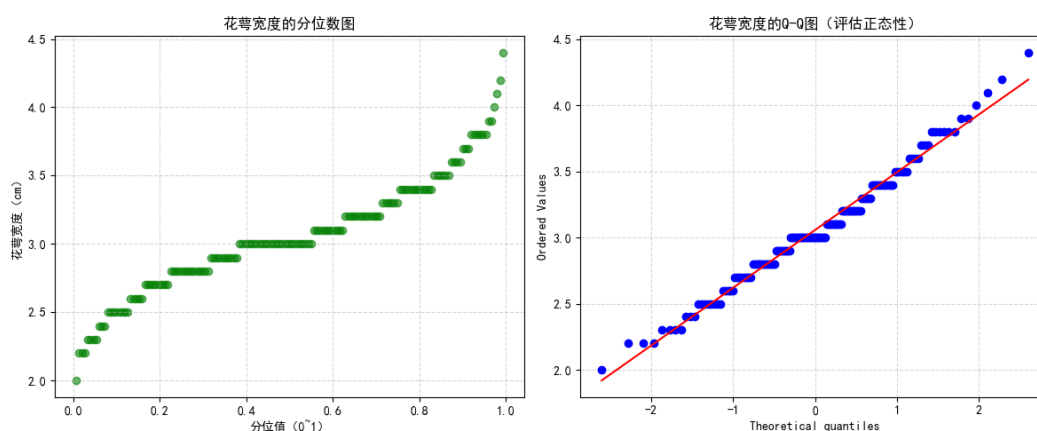
```
plt.rcParams["font.family"] = ["SimHei", "WenQuanYi Micro Hei", "Heiti TC"]
```

```
plt.rcParams["axes.unicode_minus"] = False
```

```
sepal_width = df["花萼宽度"] # 直接使用中文列名提取数据

plt.figure(figsize=(12, 5))
# 分位数图
plt.subplot(1, 2, 1)
sorted_data = sepal_width.sort_values()
n = len(sorted_data)
quantiles = [(i + 1) / (n + 1) for i in range(n)] # 计算分位值
plt.scatter(quantiles, sorted_data, color="green", alpha=0.6)
plt.xlabel("分位值 (0~1)")
plt.ylabel("花萼宽度 (cm)")
plt.title("花萼宽度的分位数图")
plt.grid(linestyle="--", alpha=0.5)
# Q-Q 图
plt.subplot(1, 2, 2)
stats.probplot(sepal_width, dist="norm", plot=plt)
plt.title("花萼宽度的 Q-Q 图 (评估正态性)")
plt.grid(linestyle="--", alpha=0.5)
plt.tight_layout()
plt.show()
```

结果:



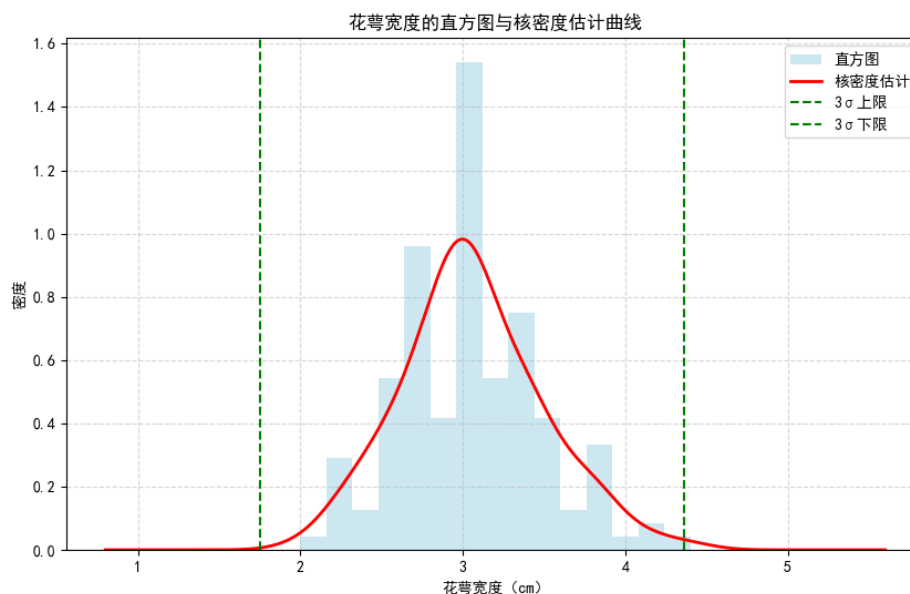
```
def detect_3sigma(series): # 定义 3σ 异常值检测函数
    """
    基于 3σ 原则检测异常值
    参数: series - 待检测的 pandas Series
    返回: 异常值的 Series (包含索引和值)
    """
    mean = series.mean() # 均值
    std = series.std() # 标准差
    upper_limit = mean + 3 * std # 上限
    lower_limit = mean - 3 * std # 下限
    outliers = series[
        (series > upper_limit) | (series < lower_limit)
    ]
```

```
    ] # 筛选出超过上下限的值（异常值）
    return outliers
sepal_width = df["花萼宽度"]
outliers_3sigma = detect_3sigma(sepal_width) # 2. 应用函数检测“花萼宽度”列的异常值
# 打印异常值结果
print(f"异常值数量: {len(outliers_3sigma)}")
if len(outliers_3sigma) > 0:
    print("异常值具体数值（索引：值）：")
    print(outliers_3sigma)
else:
    print("未检测到异常值")
print()
结果：
```

```
异常值数量: 1
异常值具体数值（索引：值）： 15    4.4
Name: 花萼宽度, dtype: float64
```

```
# 绘制“花萼宽度”的直方图与核密度估计曲线
plt.figure(figsize=(10, 6))
sepal_width.plot(
    kind="hist", bins=15, density=True, alpha=0.6, color="lightblue", label="
直方图"
) # 直方图（density=True 使纵轴为密度，与核密度曲线匹配）
sepal_width.plot(
    kind="kde", color="red", linewidth=2, label="核密度估计"
) # 核密度估计曲线
# 添加 3σ 上下限参考线
mean = sepal_width.mean()
std = sepal_width.std()
plt.axvline(mean + 3 * std, color="green", linestyle="--", label="3σ 上限")
plt.axvline(mean - 3 * std, color="green", linestyle="--", label="3σ 下限")

plt.xlabel("花萼宽度（cm）") # 添加标签和标题
plt.ylabel("密度")
plt.title("花萼宽度的直方图与核密度估计曲线")
plt.legend()
plt.grid(linestyle="--", alpha=0.5)
plt.show()
结果：
```



⑤ 使用 IQR 方法（箱线图准则）检测“花萼宽度”中的异常值：

➤ 计算 Q1、Q3、IQR，确定异常值范围；

➤ 打印异常值数量及数值；

```
Q1 = df["花萼宽度"].quantile(0.25) # 下四分位数（25%分位数）
```

```
Q3 = df["花萼宽度"].quantile(0.75) # 上四分位数（75%分位数）
```

```
IQR = Q3 - Q1 # 四分位距
```

```
lower_bound = Q1 - IQR * 1.5 # 异常值下限
```

```
upper_bound = Q3 + IQR * 1.5 # 异常值上限
```

```
wrongdatabox = df[
    (df["花萼宽度"] < lower_bound) | (df["花萼宽度"] > upper_bound)
] # 筛选异常值数据（包含所有特征和标签，便于完整查看异常样本）
```

```
print(f"下四分位数（Q1）：{Q1:.2f}")
```

```
print(f"上四分位数（Q3）：{Q3:.2f}")
```

```
print(f"四分位距（IQR）：{IQR:.2f}")
```

```
print(f"异常值判定范围：小于{lower_bound:.2f} 或 大于{upper_bound:.2f}")
```

```
print(f"检测到的异常值数量：{len(wrongdatabox)}")
```

```
print("\n 异常数据（包含所有特征和标签）：")
```

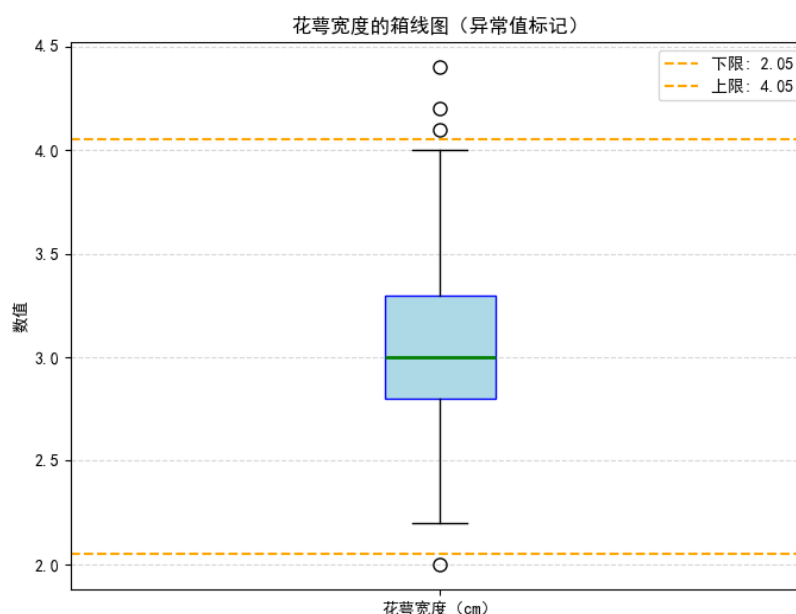
```
print(wrongdatabox) # 打印结果
```

结果：

```
异常数据（包含所有特征和标签）：
   花萼长度  花萼宽度  花瓣长度  花瓣宽度  target
15    5.7    4.4    1.5    0.4         0
32    5.2    4.1    1.5    0.1         0
33    5.5    4.2    1.4    0.2         0
60    5.0    2.0    3.5    1.0         1
```

```
下四分位数 (Q1) : 2.80
上四分位数 (Q3) : 3.30
四分位距 (IQR) : 0.50
异常值判定范围: 小于2.05 或 大于4.05
检测到的异常值数量: 4
```

```
plt.figure(figsize=(8, 6)) # 绘制箱线图可视化异常值
plt.boxplot(
    df["花萼宽度"],
    patch_artist=True,
    boxprops=dict(facecolor="lightblue", color="blue"),
    flierprops=dict(marker="o", color="red", markersize=8), # 突出显示异常值
    medianprops=dict(color="green", linewidth=2),
)
plt.xticks([1], ["花萼宽度 (cm)"])
plt.ylabel("数值")
plt.title("花萼宽度的箱线图 (异常值标记)")
plt.axhline(
    lower_bound, color="orange", linestyle="--", label=f"下限: {lower_bound:.2f}"
) # 添加下限参考线
plt.axhline(
    upper_bound, color="orange", linestyle="--", label=f"上限: {upper_bound:.2f}"
) # 添加上限参考线
plt.legend()
plt.grid(linestyle="--", alpha=0.5, axis="y")
plt.show() # 绘图
结果:
```



2. 基于卡方拟合优度检验的变压器故障原因分布分析

代码:

```
import scipy.stats as stats
```

① 输出零假设与备择假设

```
print("=== 卡方拟合优度检验假设 ===")
print(
    f"零假设 H0: 2025 年该公司 110kV 及以上变压器的各类故障原因分布比例与历史分布一  
致, 即: "
)
print(
    f"          绝缘老化占 40%、过载运行占 30%、制造缺陷占 20%、外部短路占 10%, 无系  
统性偏移。"
)
print(
    f"备择假设 H1: 2025 年该公司 110kV 及以上变压器的故障原因分布比例与历史分布不一  
致, 存在系统性偏移。"
)
print("=====\\n")
```

② 期望频数

```
observed = [
    92,
    58,
    32,
    18,
] # 2025 年实际故障次数: 绝缘老化、过载运行、制造缺陷、外部短路
expected = [80, 60, 40, 20] # 基于历史比例的期望频数 (总故障数 200)
print(f"期望频数: {expected}\\n")
```

③ 计算卡方统计量

```
chi2_stat = sum(((o - e) ** 2) / e for o, e in zip(observed, expected))
print(f"卡方统计量: {chi2_stat:.4f}\\n")
```

确定自由度 (类别数-1)

```
df = len(observed) - 1 # 4 类故障, 自由度=3
```

④ 临界值 ($\alpha=0.05$) p 值 $\alpha=0.05$ 下统计决策

```
critical_value = stats.chi2.ppf(1 - 0.05, df)
p_value = stats.chi2.sf(chi2_stat, df)
print(f"自由度: {df}\\n")
print(f" $\alpha=0.05$  时的临界值: {critical_value:.4f}\\n")
print(f"p 值: {p_value:.4f}")
if chi2_stat > critical_value or p_value < 0.05:
    print("\\n 结论: 拒绝零假设 (H0), 2025 年故障模式与历史分布存在显著差异。")
```

```
else:
    print("\n 结论： 不拒绝零假设（H0）， 2025 年故障模式与历史分布无显著差异。")
```

结果：

```
=== 卡方拟合优度检验假设 ===
零假设 H0: 2025年该公司110kV及以上变压器的各类故障原因分布比例与历史分布一致，即：
          绝缘老化占40%、过载运行占30%、制造缺陷占20%、外部短路占10%，无系统性偏移。
备择假设 H1: 2025年该公司110kV及以上变压器的故障原因分布比例与历史分布不一致，存在系统性偏移。
=====

期望频数: [80, 60, 40, 20]

卡方统计量: 3.6667

自由度: 3

α=0.05时的临界值: 7.8147

p值: 0.2998

结论： 不拒绝零假设（H0）， 2025年故障模式与历史分布无显著差异。
```

检验结果显示故障分布与历史模式无显著差异，对运维策略的启示：延续重点：优先防控绝缘老化（40%）和过载运行（30%），强化绝缘监测和负荷调度；常规防控：保持对制造缺陷（20%）和外部短路（10%）的常规检测与防护；动态关注：虽当前无偏移，但需跟踪新能源接入和负荷波动对故障模式的潜在长期影响，预留调整空间。

3. 基于卡方独立性检验的断路器运维策略与可靠性关系分析

代码：

```
import pandas as pd
import scipy.stats as stats

print()
# ① 输出假设与备择假设
print("=== 卡方独立性检验假设 ===")
print(
    f"零假设 H0: 断路器是否发生非计划停运与其所采用的运维策略相互独立（无统计关联）。"
)
print(
    f"备择假设 H1: 断路器是否发生非计划停运与其所采用的运维策略不独立（存在统计关联）。"
)
print("=====\\n")
```

结果：

```
=== 卡方独立性检验假设 ===
零假设 H0: 断路器是否发生非计划停运与其所采用的运维策略相互独立（无统计关联）。
备择假设 H1: 断路器是否发生非计划停运与其所采用的运维策略不独立（存在统计关联）。
=====
```


② 构建列联表

```
data = {"发生非计划停运": [45, 20], "未发生非计划停运": [105, 130]}
contingency_table = pd.DataFrame(data, index=["传统定期检修", "预测性维护"])
print("=== 列联表 ===")
print(contingency_table, "\n")
```

结果:

```
=== 列联表 ===
              发生非计划停运  未发生非计划停运
传统定期检修             45             105
预测性维护               20             130
```

③ 计算期望频数矩阵与卡方统计量

```
row_totals = contingency_table.sum(axis=1) # 每行总样本数
col_totals = contingency_table.sum(axis=0) # 每列总样本数
total = contingency_table.sum().sum() # 总样本数

expected = [] # 构建期望频数矩阵
for i in range(len(row_totals)):
    row = []
    row_total = row_totals.iloc[i] # 用 iloc 按位置访问, 避免索引警告
    for j in range(len(col_totals)):
        col_total = col_totals.iloc[j]
        exp_val = (row_total * col_total) / total
        row.append(exp_val)
    expected.append(row) # 将每行数据添加到列表
expected_df = pd.DataFrame(
    expected, index=contingency_table.index, columns=contingency_table.columns
) # 转换为 DataFrame
print("=== 期望频数矩阵 ===")
print(expected_df.round(2), "\n") # 保留 2 位小数
chi2_stat = ((contingency_table - expected_df) ** 2 / expected_df).sum().sum()
print(f"卡方统计量: {chi2_stat:.4f}\n") # 计算输出卡方统计量
```

结果:

```
=== 期望频数矩阵 ===
              发生非计划停运  未发生非计划停运
传统定期检修             32.5             117.5
预测性维护               32.5             117.5

卡方统计量: 12.2750
```

④ 卡方独立性检验 ($\alpha=0.05$)

```
df = (contingency_table.shape[0] - 1) * (contingency_table.shape[1] - 1) # 自由度
```

```
critical_value = stats.chi2.ppf(1 - 0.05, df) # 临界值
p_value = stats.chi2.sf(chi2_stat, df) # p 值
```

```
print(f"自由度: {df}")
print(f" $\alpha=0.05$  时的临界值: {critical_value:.4f}")
print(f"p 值: {p_value:.6f}\n")
```

结果:

```
自由度: 1
 $\alpha=0.05$ 时的临界值: 3.8415
p值: 0.000459
```

统计决策

```
if chi2_stat > critical_value or p_value < 0.05:
    print("结论: 拒绝零假设 ( $H_0$ ), 运维策略与非计划停运存在统计关联。")
    # 计算可靠率, 比较未发生非计划停运的比例
    traditional_ratio = (
        contingency_table.loc["传统定期检修", "未发生非计划停运"]
        / row_totals["传统定期检修"]
    )
    predictive_ratio = (
        contingency_table.loc["预测性维护", "未发生非计划停运"]
        / row_totals["预测性维护"]
    )
    print(f"传统定期检修未发生停运比例: {traditional_ratio:.2%}")
    print(f"预测性维护未发生停运比例: {predictive_ratio:.2%}")
    print("因此, 预测性维护在可靠性上更具优势。")
else:
    print("结论: 不拒绝零假设 ( $H_0$ ), 即运维策略与非计划停运无统计关联。")
```

结果:

```
结论: 拒绝零假设 ( $H_0$ ), 运维策略与非计划停运存在统计关联。
传统定期检修未发生停运比例: 70.00%
预测性维护未发生停运比例: 86.67%
因此, 预测性维护在可靠性上更具优势。
```